

A Practical Guide to DBSCAN

TOPIC -- DBSCAN

-- 01 --

Advantages DBSCAN does not require one to specify the number of clusters in the data a priori, as opposed to k-means. DBSCAN can find arbitrarily-shaped clusters. It can even find a cluster completely surrounded by (but not connected to) a different cluster. Due to the MinPts parameter, the so-called single-link effect (different clusters being connected by a thin line of points) is reduced. DBSCAN has a notion of noise, and is robust to outliers. DBSCAN requires just two parameters and is mostly insensitive to the ordering of the points in the database. (However, points sitting on the edge of two different clusters might swap cluster membership if the ordering of the points is changed, and the cluster assignment is unique only up to isomorphism.) DBSCAN is designed for use with databases that can accelerate region queries, e.g. using an R* tree. The parameters minPts and ϵ can be set by a domain expert, if the data is well understood.

-- 02 --

$$\min_{C \subset C, \forall p, q \in C, d(p, q) \leq \epsilon} |C| \quad \forall C_i \in C, |C_i| \geq \text{minPts}$$

-- 03 --

History In 1972, Robert F. Ling published a closely related algorithm in "The Theory and Construction of k-Clusters" in The Computer Journal with an estimated runtime complexity of $O(n^3)$. DBSCAN has a worst-case of $O(n^2)$, and the database-oriented range-query formulation of DBSCAN allows for index acceleration. The algorithms slightly differ in their handling of border points.

-- 04 --

A point p is a core point if at least minPts points are within distance ϵ of it (including p). A point q is directly reachable from p if point q is within distance ϵ from core point p . Points are only said to be directly reachable from core points. A point q is reachable from p if there is a path $p_1, \dots, p_n$ with $p_1 = p$ and $p_n = q$, where each p_{i+1} is directly reachable from p_i . Note that this implies that the initial point and all points on the path must be core points, with the possible exception of q . All points not reachable from any other point are outliers or noise points. Now if p is a core point, then it forms a cluster together with all points (core or

non-core) that are reachable from it. Each cluster contains at least one core point; non-core points can be part of a cluster, but they form its "edge", since they cannot be used to reach more points.

-- 05 --

Original query-based algorithm DBSCAN requires two parameters: ϵ (eps) and the minimum number of points required to form a dense region (minPts). It starts with an arbitrary starting point that has not been visited. This point's ϵ -neighborhood is retrieved, and if it contains sufficiently many points, a cluster is started. Otherwise, the point is labeled as noise. Note that this point might later be found in a sufficiently sized ϵ -environment of a different point and hence be made part of a cluster. If a point is found to be a dense part of a cluster, its ϵ -neighborhood is also part of that cluster. Hence, all points that are found within the ϵ -neighborhood are added, as is their own ϵ -neighborhood when they are also dense. This process continues until the density-connected cluster is completely found. Then, a new unvisited point is retrieved and processed, leading to the discovery of a further cluster or noise. DBSCAN can be used with any distance function (as well as similarity functions or other predicates). The distance function (dist) can therefore be seen as an additional parameter. The algorithm can be expressed in pseudocode as follows:

-- 06 --

Reachability is not a symmetric relation: by definition, only core points can reach non-core points. The opposite is not true, so a non-core point may be reachable, but nothing can be reached from it. Therefore, a further notion of connectedness is needed to formally define the extent of the clusters found by DBSCAN. Two points p and q are density-connected if there is a point o such that both p and q are reachable from o . Density-connectedness is symmetric. A cluster then satisfies two properties:

-- 07 --

Availability Different implementations of the same algorithm were found to exhibit enormous performance differences, with the fastest on a test data set finishing in 1.4 seconds, the slowest taking 13803 seconds. The differences can be attributed to implementation quality, language and compiler differences, and the use of indexes for acceleration.

-- 08 --

Relationship to spectral clustering A spectral implementation of DBSCAN is related to spectral clustering in the trivial case of determining connected graph

components — the optimal clusters with no edges cut. However, it can be computationally intensive, up to $O(n^3)$. Additionally, one has to choose the number of eigenvectors to compute. For performance reasons, the original DBSCAN algorithm remains preferable to its spectral implementation.

-- 09 --

Density-based spatial clustering of applications with noise (DBSCAN) is a data clustering algorithm proposed by Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu in 1996. It is a density-based clustering non-parametric algorithm: given a set of points in some space, it groups together points that are closely packed (points with many nearby neighbors), and marks as outliers points that lie alone in low-density regions (those whose nearest neighbors are too far away). DBSCAN is one of the most commonly used and cited clustering algorithms. In 2014, the algorithm was awarded the Test of Time Award (an award given to algorithms which have received substantial attention in theory and practice) at the leading data mining conference, ACM SIGKDD. As of July 2020, the follow-up paper "DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN" appears in the list of the 8 most downloaded articles of the prestigious ACM Transactions on Database Systems (TODS) journal. Another follow-up, HDBSCAN*, was initially published by Ricardo J. G. Campello, David Moulavi, and Jörg Sander in 2013, then expanded upon with Arthur Zimek in 2015. It revises some of the original decisions such as the border points, and produces a hierarchical instead of a flat result.